

## Session 10 — Study Guide: Exception Filters

### Where the session ends

---

Every field decorated with `@Transform(trim)` in the app — on both Create DTOs, inherited into both Update DTOs via `PartialType` — is guarded against non-string input, so a bad request gets a clean `400` instead of the wrong status code. On top of that, a global `AllExceptionsFilter` gives the application its own control over how unexpected errors are reported: a consistent response shape `( path / timestamp / method )` and a single place for the app's own logging.

### What NestJS already does, before any of this

---

This is the single most important thing to understand before anything else in this session: NestJS already has a built-in default exception handler, running on every request, with zero custom code. Before this session's own filter exists, a non-string `name` already produces, for the client, exactly `{"statusCode":500,"message":"Internal server error"}` — never the raw error, never a stack trace — and the real error is already logged server-side. **Nothing in this app has ever leaked an internal error to a client.** The real problem this session solves is different: `500` is the WRONG status code for bad input, and the application has no control over the shape of its own unexpected errors. Keep this distinction in mind for everything below.

### The real bug this session closes

---

`@Transform(({ value }) => value.trim())` assumes `value` is always a string. `class-transformer`'s transform phase runs BEFORE `class-validator`'s own checks — so `POST /projects` with `{"name": 123}` calls `.trim()` on a number, throws a real `TypeError`, and Nest's own default handling (above) turns that into a safe `500` for the client. The result: a `500` instead of the `400` the bad input actually deserves.

### Step one: fix it at the source

---

TS

```
@Transform(({ value }) => (typeof value === 'string' ? value.trim() : value))
```

`typeof value === 'string'` only calls `.trim()` when that's actually safe. Anything else passes through unchanged — straight to `@IsString()`, which now rejects it cleanly with a `400`. This is the complete fix for this bug. Nothing else is needed to close it.

### Step two: why build a filter at all, if Nest already protects us?

---

Fixing `CreateProjectDto` does nothing for `CreateWorkspaceDto` — it has the identical unguarded `.trim()` calls. And NestJS's own default handling, while safe, gives the application no control over the response SHAPE, or where errors get logged. We want something that lets the application decide those things for itself — not a tool to "catch what would otherwise leak," because nothing leaks either way.

## Step three: a global exception filter — a customization layer, not a safety net

TS

```
@Catch()
export class AllExceptionsFilter implements ExceptionFilter {
  private readonly logger = new Logger(AllExceptionsFilter.name);

  catch(exception: unknown, host: ArgumentsHost) {
    const ctx = host.switchToHttp();
    const response = ctx.getResponse<Response>();
    const request = ctx.getRequest<Request>();

    if (exception instanceof HttpException) {
      response.status(exception.getStatus()).json(exception.getResponse());
      return;
    }

    this.logger.error(exception);
    response.status(HttpStatus.INTERNAL_SERVER_ERROR).json({
      statusCode: HttpStatus.INTERNAL_SERVER_ERROR,
      message: 'Internal server error',
      path: request.url,
    });
  }
}
```

`@Catch()` with no argument intercepts everything thrown during any request, anywhere. Known errors (already `HttpException` — `404`, `409`, `400` from validation) pass through exactly as they already were — this app's own existing behavior for those is not replaced, just preserved. Anything else gets logged for the developer and reported to the client with the application's OWN chosen shape, starting with `path`.

TS

```
app.useGlobalFilters(new AllExceptionsFilter());
```

One line in `main.ts`, the same shape as the global `ValidationPipe` this app already registers.

## Common beginner mistakes

- `value => value.trim()` instead of `({ value }) => ...`. The callback receives one options object, not the raw value — the Session 09 mistake, still possible here.
- **Saying "the filter exists so errors don't leak."** Nothing leaks, with or without the filter — Nest's own default handling already prevents that. The filter is about customization and control.
- **Forgetting `instanceof HttpException`**. Without it, every already-correct `404` / `409` in the app gets swallowed into the filter's own generic shape too.

- **Building the filter without registering it.** `@Catch()` alone does nothing — only `app.useGlobalFilters(...)` turns it on.

## Not yet

---

These exist in NestJS, but are deliberately **not** covered this session:

- Guards and Interceptors (later, once there's something worth guarding)
- Custom decorators
- `@nestjs/config` / `.env`
- Swagger/OpenAPI
- Any database, authentication, or Next.js
- `PickType` / `OmitType` (Session 09's own FLEX, not repeated here)

## Glossary

---

- **ExceptionHandler** — an interface with one method, `catch(exception, host)`, implemented by a class that wants to intercept unhandled exceptions.
- **ArgumentsHost** — a context-agnostic wrapper around the current request; `switchToHttp()` narrows it to the familiar Express `request` / `response` pair.
- **HttpException** — the base class every one of Nest's own built-in exceptions extends; already carries a correct status and a safe response body.
- **Type guard** — a runtime check (`typeof value === 'string'`) that narrows what can be assumed about a value before acting on it.

## Review questions

---

1. Before any of this session's own code exists, what does NestJS already do with an unexpected error — for the client, and server-side?
2. Why is `500` the wrong response for a non-string `name`, and what is the actual fix?
3. What would happen to an ordinary `404` if the filter didn't check `instanceof HttpException`?
4. The type guard and the global filter both exist in this session. Why keep both, if the filter isn't needed for safety?

## Answers

---

1. It already returns the client a safe `{ "statusCode": 500, "message": "Internal server error" }` and already logs the real error server-side — automatically, with zero custom code.
2. `500` means the server is at fault; here the client sent bad input, which should be `400`. The fix is a type guard inside `@Transform`'s own callback, so `@IsString()` gets the chance to reject it correctly.
3. It would be swallowed into the filter's own generic `500` response, losing its real, already- correct status and message.

4. The type guard is the correct, complete fix for the one specific bug. The filter is a separate concern — customization and control over how the app reports whatever it DOESN'T have a specific fix for, today or in a future session. Neither replaces the other.

## What you should be able to explain now

---

- What NestJS already does, by default, with an error nobody specifically handled.
- Why `class-transformer` running before `class-validator` can turn a bad request into the wrong status code — not a crash that reaches the client.
- Why this session built a specific fix and a general customization layer, and why neither one is about preventing a leak.